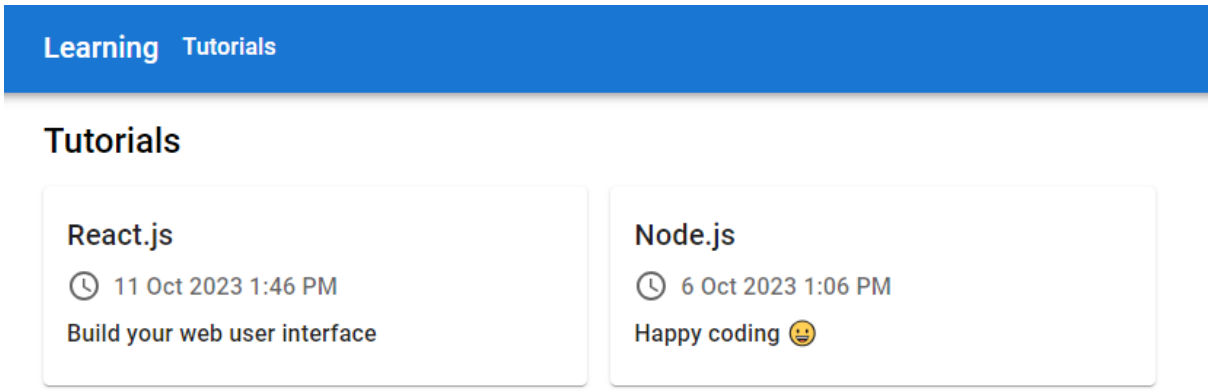
Lab 3 React Web App

Learning Outcomes

- Create client web app using React.js
- Use MUI components for application
- Add new page to list tutorials
- Use state to display tutorial list



Note: Please work on Lab 3 based on **Lab 2b** solution using MySQL.

Activity 1: Create client web app using React.js

1. Use VS Code to open the folder **learning-app**.

2. Add a new folder **client** under learning-app.

3. Open terminal, change directory to client folder.
   cd client

4. Run command to create the client app:
   npm create vite .
   Note: Remember to put "." to indicate it's for the current directory.
   Vite is a rapid development tool for modern web projects. When you run this command the first time, it will ask you to install vite, just enter 'Y' to continue.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

5. Select 'React' for the framework:

```
? Select a framework: » - Use arrow-keys. Return to submit.
    Vanilla
    Vue
>   React
    Preact
    Lit
    Svelte
    Others
```

React.js is a JavaScript framework for building user interfaces.
To learn more about React.js, can see the link https://reactjs.org

6. Select 'JavaScript + SWC' for the variant:

```
√ Select a framework: » React
? Select a variant: » - Use arrow-keys. Return to submit.
    TypeScript
    TypeScript + SWC
    JavaScript
>   JavaScript + SWC
```

SWC stands for Speedy Web Compiler.

7. Observe the folders and files auto generated:
   - Folder **public**: for the static files
   - Folder **src**: for the JavaScript source code
   - File **index.html**: starting html for the app
   - File **package.json**: already includes some scripts and react dependencies
   - File **vite.config.js**: vite configuration

8. Edit file index.html to set the page title:

```
<head>
  …
  <title>Learning App</title>
</head>
```

9. Edit file package.json to add start script:

```
"scripts": {
  "start": "vite",
  …
},
```


Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

10. Edit file vite.config.js to set the port number:

```
export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000
  }
})
```

11. Run command to install packages:
    npm i

12. Run command to start client:
    npm start

13. Click on the link in the terminal to open url http://localhost:3000

Hot Module Replacement (HMR) exchanges, adds, or removes modules while an application is running, without a full reload. The app will auto reload after your changes are saved.

14. In VS Code, open file App.jsx, edit the HTML text and save.
    JSX stands for JavaScript XML. JSX allows us to write HTML in React. JSX makes it easier to write and add HTML in React.

15. In the browser, observe that the page is auto refreshed to show the changes.

16. Press **Ctrl + C** to stop the client.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Activity 2: Use MUI components for application

1. Run command to install packages for MUI:
   npm i @mui/material@5 @emotion/react@11 @emotion/styled@11
   MUI is a massive library of UI components used to build React applications.
   To learn more about MUI, can see the link https://mui.com

2. Run command to install package react-router-dom:
   npm i react-router-dom
   The react-router-dom package contains bindings for using React Router in web applications.

3. Open src/App.jsx, **replace all** the existing code as below:

```
import './App.css';
import { Container, AppBar, Toolbar, Typography } from '@mui/material';
import { BrowserRouter as Router, Routes, Route, Link }
from 'react-router-dom';

function App() {
  return (
    <Router>
      <AppBar position="static" className='AppBar'>
        <Container>
          <Toolbar disableGutters={true}>
            <Link to="/">
              <Typography variant="h6" component="div">
                Learning
              </Typography>
            </Link>
            <Link to="/tutorials" ><Typography>Tutorials</Typography></Link>
          </Toolbar>
        </Container>
      </AppBar>

      <Container>
        <Routes>
          <Route path={"/"} />
          <Route path={"/tutorials"} />
        </Routes>
      </Container>
    </Router>
  );
}

export default App;
```


Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

App.jsx is the main application file. It's a component used in **main.jsx**.

Inside **Router**, we put the AppBar component with 2 links inside for all pages.
Inside **Routes**, now there are 2 routes, we will add the page element for the routes later.

4. Open src/index.css, **replace all** the existing styles as below:

```
:root {
  font-family: Inter, system-ui, Avenir, Helvetica, Arial, sans-serif;
  line-height: 1.5;
  font-weight: 400;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  -webkit-text-size-adjust: 100%;
}

body {
  margin: 0;
}
```

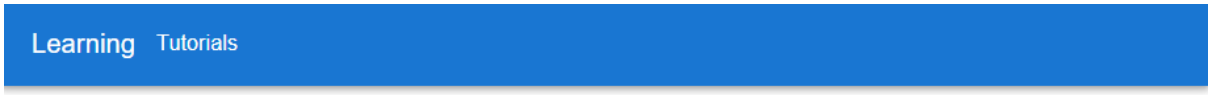
index.css is used in main.jsx.

5. Open src/App.css, **replace all** the existing styles as below:

```
.AppBar a {
  color: inherit;
  text-decoration: none;
  margin-right: 16px;
}
```

App.css is used in App.jsx. The styles are for the Link component inside AppBar.

6. Run the client to see the app bar. The page below the app bar is empty now.

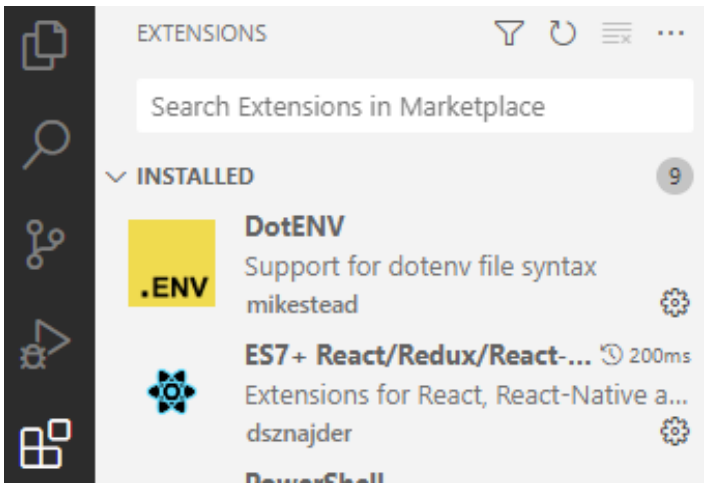


Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Activity 3: Add new page to list tutorials

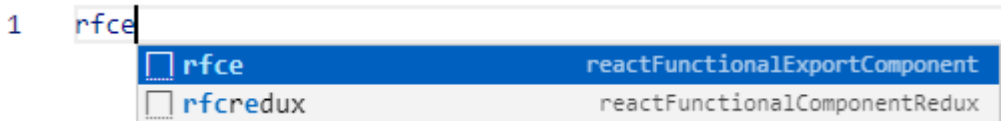
1. Under the folder src, add a new folder "**pages**"

2. Under Extensions, install **ES7+ React/Redux/React-Native snippets**.
   It's used to generate node snippets for React components.



3. Under the folder pages, add a new file **Tutorials.jsx**

4. In Tutorials.jsx, type **rfce** and enter. The code snippet reactFunctionalExportComponent is added.




```
import React from 'react'

function Tutorials() {
  return (
    <div>Tutorials</div>
  )
}

export default Tutorials
```

The auto generated component renders a simple div with the file name.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

5. Open App.jsx, add the Tutorials element for the 2 routes:

```
import Tutorials from './pages/Tutorials';
…

        <Routes>
          <Route path={"/"} element={<Tutorials />} />
          <Route path={"/tutorials"} element={<Tutorials />} />
        </Routes>
…
```

6. In the browser, observe that the page Tutorials is loaded.

7. In VS Code, open Tutorials.jsx, change the code to use MUI components:

```
import React from 'react';
import { Box, Typography } from '@mui/material';

function Tutorials() {
    return (
        <Box>
            <Typography variant="h5" sx={{ my: 2 }}>
                Tutorials
            </Typography>
        </Box>
    );
}

export default Tutorials;
```

Box serves as a wrapper component, like the html div.
Typography is used to display text, the prop variant applies the theme typography styles.
The sx prop is a shortcut for defining custom styles.

8. In the browser, observe that the style for the title is updated.

9. Press **Ctrl + C** to stop the client.

10. Run command to install package axios:
    npm i axios
    Axios is a promise-based HTTP client for the browser and node.js.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

11. In Terminal, click the icon Split Terminal.

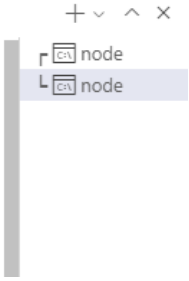
12. In the new terminal, change directory to server folder and start server:

```
Microsoft Windows [Version 10.0.19044.2486]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user\Documents\GitHub\fullstack\practical_04\learning-app>cd server

C:\Users\user\Documents\GitHub\fullstack\practical_04\learning-app\server>npm start

> server@1.0.0 start
> nodemon index.js
```



13. Under the folder client, add a new file .env:

```
VITE_API_BASE_URL = "http://localhost:3001"
```

In vite web app, the environment variable name must start with **VITE_**.
You don't need to install the package dotenv.

14. Under the folder src, add a new file http.js:

```
import axios from "axios";

const instance = axios.create({
    baseURL: import.meta.env.VITE_API_BASE_URL
});

export default instance;
```

The API base URL is defined in the axios instance.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

15. In Tutorials.jsx, add code to call api:

```
import React, { useEffect } from 'react';
import { Box, Typography } from '@mui/material';
import http from '../http';

function Tutorials() {
    useEffect(() => {
        http.get('/tutorial').then((res) => {
            console.log(res.data);
        });
    }, []);

    return (
    …
    );
}

export default Tutorials;
```

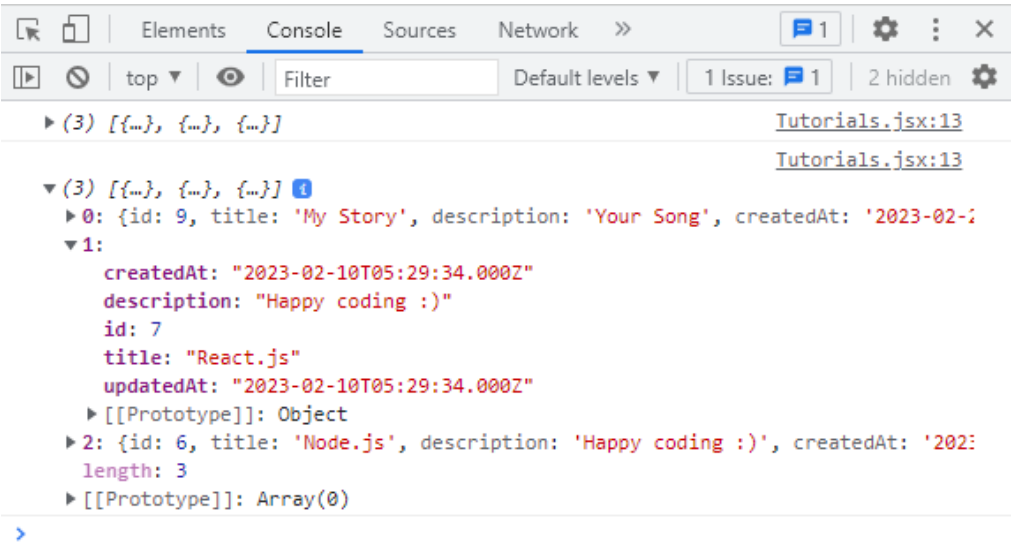
A Hook is a special function that lets you “hook into” React features.

The Effect Hook lets you perform side effects in function components. By using the Effect Hook, you tell React that your component needs to do something after rendering.

In the function useEffect(), the 1st parameter defines the function to be triggered.
The 2nd parameter indicates when the function will be triggered. The empty array [] means the function will be triggered when the page is loaded.

The http client calls the server api get method with the url to get the tutorial list and log the response data in the browser console.

16. In the browser, right click any place in the web page and select the **Inspect** option. It will open the developer tools, select **Console** to view the logs:



Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Activity 4: Use state to display tutorial list

1. In VS Code, open Tutorials.jsx, add code to use state for tutorial list:

```
import React, { useEffect, useState } from 'react';
…

function Tutorials() {
    const [tutorialList, setTutorialList] = useState([]);

    useEffect(() => {
        http.get('/tutorial').then((res) => {
            console.log(res.data);
            setTutorialList(res.data);
        });
    }, []);

    …
}
```

The hook useState lets you add React state to function components.
The variable **tutorialList** is a state variable.
The function **setTutorialList** is the setter function for the state variable.
The initial value of the state variable is an empty array **[]**.
After getting the api response, call setTutorialList() to set the value of tutorialList.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

2. Add code to display tutorial list using grid and card:

```
import { Box, Typography, Grid, Card, CardContent } from '@mui/material';
…

    return (
        <Box>
            <Typography variant="h5" sx={{ my: 2 }}>
                Tutorials
            </Typography>

            <Grid container spacing={2}>
                {
                    tutorialList.map((tutorial, i) => {
                        return (
                            // code_to_be_added…
                        );
                    })
                }
            </Grid>
        </Box>
    );
```

The responsive layout **Grid** adapts to screen size and orientation, ensuring consistency across layouts. There are two types of layout: **container** and **item**.
In the grid container, we loop through the array using the map() function. For each object, it returns the grid item.

The code to be added is as below:

```
<Grid item xs={12} md={6} lg={4} key={tutorial.id}>
    <Card>
        <CardContent>
            <Typography variant="h6" sx={{ mb: 1 }}>
                {tutorial.title}
            </Typography>
            <Typography sx={{ whiteSpace: 'pre-wrap' }}>
                {tutorial.description}
            </Typography>
        </CardContent>
    </Card>
</Grid>
```

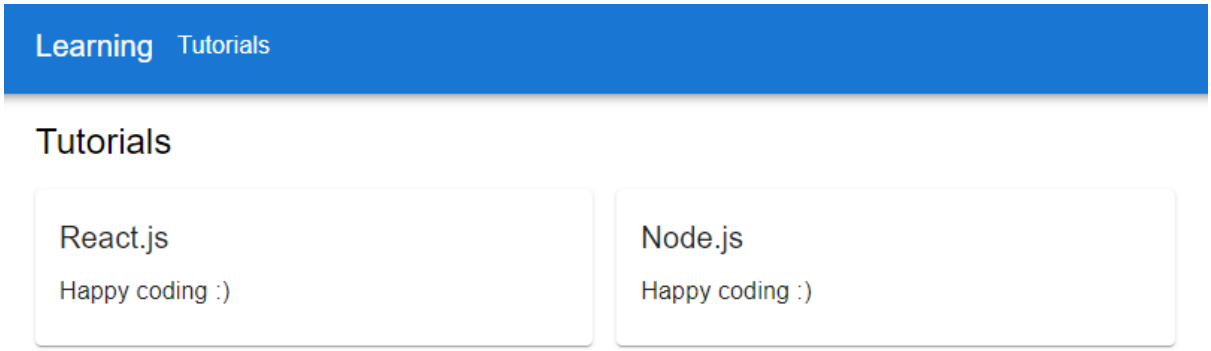
For the **Grid** item, there are five grid breakpoints: xs, sm, md, lg, and xl. Integer values can be given to each breakpoint, indicating how many of the 12 available columns are occupied.

Card contains content about a single subject.

The style *whiteSpace: 'pre-wrap'* means the whitespaces are not collapsed into one so that the line breaks will be displayed, the text also wraps to the next line when needed.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

3. In the browser, observe that the tutorials are listed in cards, with title and description.



4. In VS Code, press Ctrl + C to stop the client.

5. Run command to install package material icons:
   npm i @mui/icons-material@5
   The package includes the official Material Icons converted to SvgIcon components.
   To find out more, can see the link https://mui.com/material-ui/material-icons

6. In file Tutorials.jsx, add code to display created time below title:

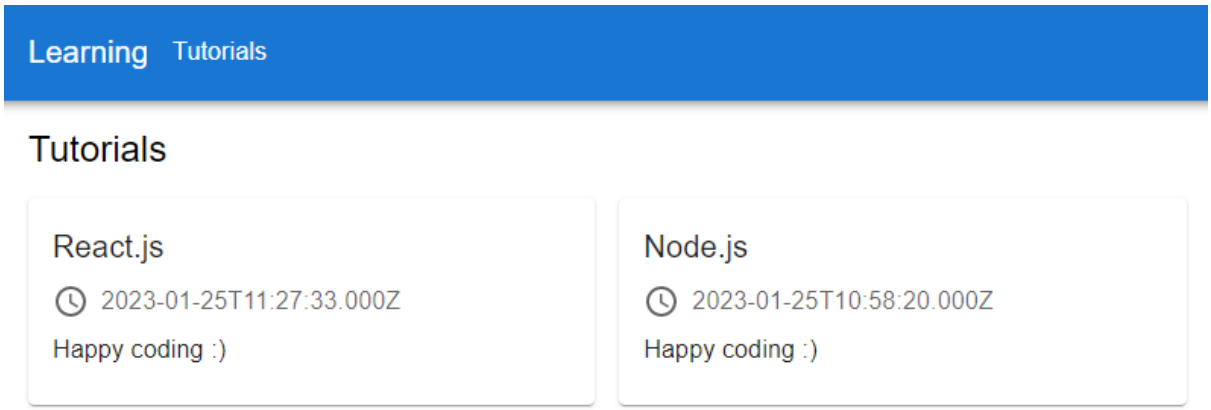
```
import { AccessTime } from '@mui/icons-material';
…
        <CardContent>
            <Typography variant="h6" sx={{ mb: 1 }}>
                {tutorial.title}
            </Typography>
            <Box sx={{ display: 'flex', alignItems: 'center', mb: 1 }}
                color="text.secondary">
                <AccessTime sx={{ mr: 1 }} />
                <Typography>
                    {tutorial.createdAt}
                </Typography>
            </Box>
            <Typography>
                {tutorial.description}
            </Typography>
        </CardContent>
```

The Box uses flex display in row direction and align items in vertical center.
The icon AccessTime is used, at the left of created time.

7. Run command to start the client.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

8. In the browser, observe that the icon and time are displayed, but the time is not in a nice format.



9. In VS Code, press Ctrl + C to stop the client.

10. Run command to install package dayjs:
    npm i dayjs
    Day.js is a library that parses, validates, manipulates, and displays dates and times.

11. Under the folder src, add new file global.js:

```
const global = {
    datetimeFormat: 'D MMM YYYY h:mm A'
}

export default global;
```

New global variable is added for the date time format.

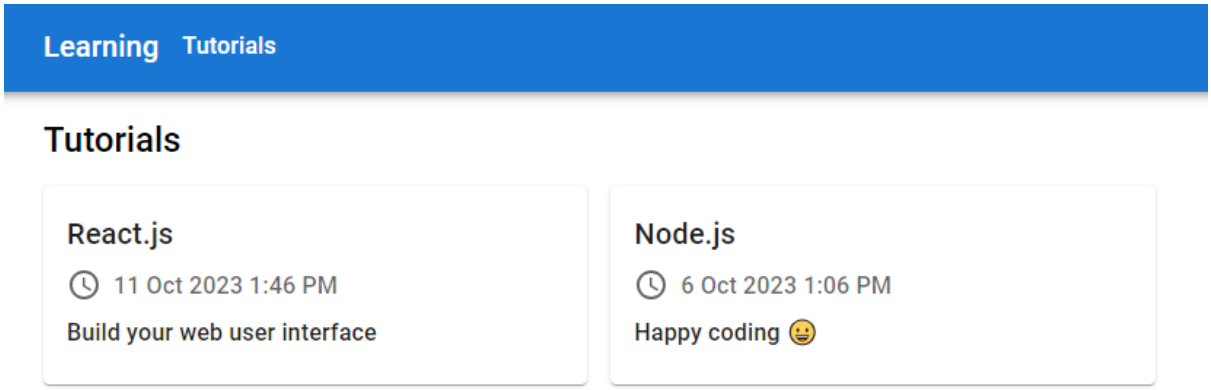
12. In file Tutorials.jsx, change code to display time in the format:

```
import dayjs from 'dayjs';
import global from '../global';
…
<AccessTime sx={{ mr: 1 }} />
<Typography>
    {dayjs(tutorial.createdAt).format(global.datetimeFormat)}
</Typography>
```

13. Run command to start the client.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

14. In the browser, observe that the time now is in a nice format.



Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.